Informe de Ejecución de Pruebas

Automatización de APIs – Billetera Digital

Proyecto	Evaluación de Automatización de APIs – Billetera Digital
Sistema bajo prueba	http://134.209.211.10:5173/ (microservicios Auth 4001, Wallet 4002, Transaction 4003, Admin 4004)
Herramientas	Karate Framework 1.4.1 · JUnit 5 · Maven · Java 17
Tipo de prueba	Regresión funcional E2E de API
Modo de ejecución	Secuencial, 1 hilo (parallel(1))
Entorno	qa
Fecha de ejecución	2026-10-28 07:37:56 p. m.
Elaborado por	Jonathan Castañeda – QA Automation Engineer
Resultado	**APROBADO**

1. Resultado de ejecución – Resumen

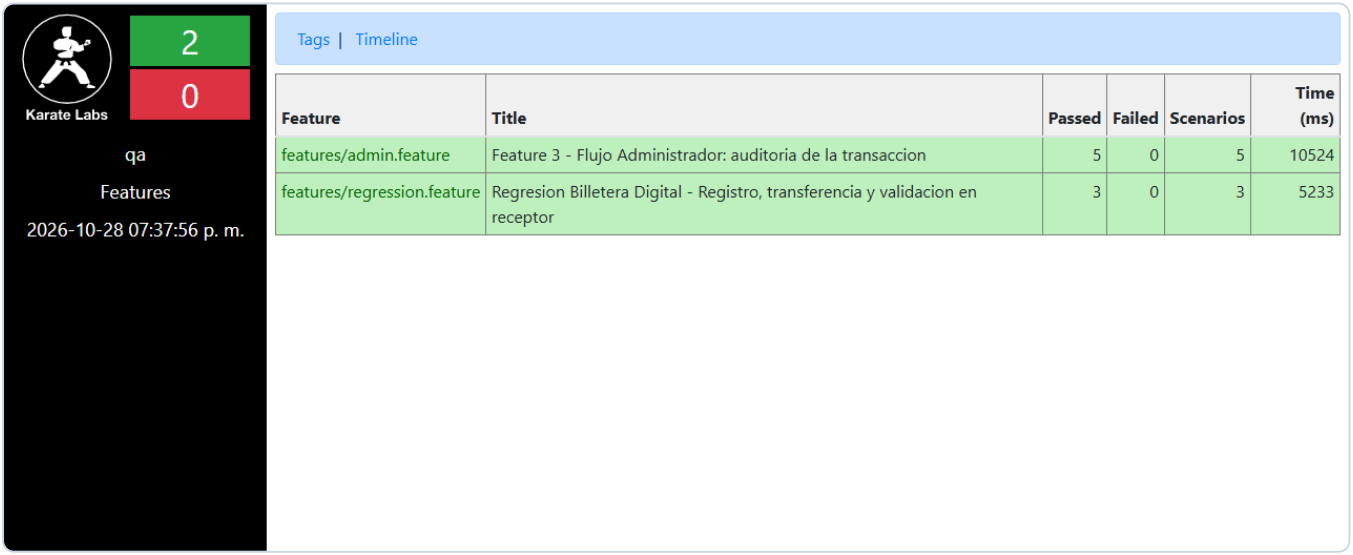
2	8	8	0	100%	15.8s
Features	Escenarios	Aprobados	Fallidos	Éxito	Duración

El flujo **Registro → Transferencia → Validación en Receptor → Auditoría del Administrador** se ejecutó de punta a punta. Los datos se comparten entre escenarios mediante un almacén global definido en `karate-config.js`, por lo que la suite corre en un solo hilo y en orden.

Datos de la ejecución

Dato	Valor
Emisor (teléfono / id)	959477034 / u_1793234259588
Receptor (teléfono / id)	959491530 / u_1793234259834
Saldo asignado a cada usuario	2,000.00 USD
Monto transferido / nota	450.00 USD / "Pago de Servicios"
transactionId	tx_1793234263209
Código de referencia	REF-449329
Saldo final Emisor	1,550.00 USD (2,000.00 − 450.00)
Saldo final Receptor	2,450.00 USD (2,000.00 + 450.00)

Reporte de Karate (resumen)



2. Resultado por escenario

#	Feature	Escenario	Validaciones	Tiempo	Estado

1	regression.feature	Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado	Registro 201 de Emisor y Receptor con teléfono aleatorio único; esquema del usuario con matchers (#string, #regex, #array); **negativo**: registro duplicado → 409 AUTH_006; abono de 2000.00 vía Admin; saldo consultado con el token de cada usuario = saldo asignado.	3.74s	**PASS**
2	regression.feature	Escenario 2 - Transferencia del Emisor al Receptor por 450.00	Login del Emisor y captura del token Bearer; transferencia de 450.00 con nota "Pago de Servicios" → 200; tabla de aserción (status COMPLETED, mensaje, transactionId, referenceCode); saldo Emisor = saldo inicial − 450.00.	0.80s	**PASS**
3	regression.feature	Escenario 3 - Validacion del saldo y del historial en el Receptor	Login del Receptor; saldo = saldo inicial + 450.00; el historial contiene el código de la transacción como INCOME de 450.00 con la contraparte y la nota correctas.	0.69s	**PASS**
4	admin.feature	Login de administrador con credenciales validas e invalidas	Login admin 200 (token Bearer, expiresIn 14400); credenciales inválidas → 401 AUTH_003.	0.44s	**PASS**
5	admin.feature	El administrador valida la transaccion en la auditoria global	La auditoría global contiene la referencia con 2 asientos: débito EXPENSE del Emisor y crédito INCOME del Receptor por 450.00.	1.52s	**PASS**
6	admin.feature	El administrador confirma los saldos finales de Emisor y Receptor	GET /admin/v1/users: Emisor y Receptor ACTIVE con los saldos finales esperados.	7.68s	**PASS**
7	admin.feature	Acceso a la auditoria sin permisos de administrador - sin token	Sin token → 401 ADM_001.	0.44s	**PASS**
8	admin.feature	Acceso a la auditoria sin permisos de administrador - token de usuario sin rol	Token de usuario sin rol admin → 403 ADM_002.	0.44s	**PASS**

Cobertura de los criterios de evaluación

Criterio	Cómo se cubre en el proyecto
1. Uso de variables correctas	Variables globales en `karate-config.js` (URLs, monto, nota, saldo, credenciales). Almacén global `store` para compartir usuarios, saldos iniciales, token y código de transacción entre escenarios y features. Variables locales con nombres descriptivos (`emisor`, `tokenEmisor`, `saldoInicialReceptor`, `codigoTransaccion`).

2. Validaciones correctas	Códigos de estado, esquemas con matchers (`#string`, `#number`, `#notnull`, `#regex`, `#array`, `##string`), `match each`, `contains`, tabla de aserciones, validación matemática de saldos, casos negativos (409, 401, 403).
3. Reporte correcto	Reporte HTML nativo de Karate con request/response, reporte PDF generado por el runner, Cucumber JSON y este informe con evidencia por escenario.
4. Código legible, ordenado y buenas prácticas	Background con configuración común, features reutilizables (`common/`) invocados con `call`, tags por escenario, comentarios por bloque, ejecución secuencial explícita, datos de prueba generados en cada corrida (sin dependencias de datos previos).

3. Detalle y evidencia de la ejecución por escenario

Para cada escenario del enunciado se muestra cada paso solicitado, cómo se implementó en Karate y el valor obtenido en esta ejecución; luego, las peticiones y respuestas HTTP registradas por Karate (los tokens se muestran truncados).

Escenario 1: Registro de Usuarios PASS

Paso del enunciado	Implementación (Karate)	Evidencia de esta ejecución
1. Crear un usuario Emisor y otro Receptor	`POST /auth/v1/register` por cada usuario; `Then status 201` y `match response.user == esquemaUsuario`.	Emisor 959477034 (u_1793234259588) Receptor 959491530 (u_1793234259834) Ambos 201
1a. Función para generar números celulares aleatorios	`generarCelular()` en `karate-config.js`: 9 dígitos que empiezan con 9 (timestamp + aleatorio), sin repetir en la ejecución; se valida con `#regex 9\d{8}`.	959477034 y 959491530
1b. Alojar los usuarios en variables globales	`store.put('emisor', emisor)` y `store.put('receptor', receptor)`; los escenarios 2, 3 y el Feature 3 los leen con `store.get(...)`.	Usuarios disponibles en los escenarios siguientes
2. Asignar saldo suficiente y validar que sea correcto	`POST /admin/v1/balance` (CREDIT) para ambos vía `call` en bucle; `match response.newBalance == amount` y, con el token de cada usuario, `GET /wallet/v1/balance` → `match response.balance == saldoAsignado`.	newBalance 2,000.00 y balance 2,000.00 para ambos
3. Guardar los saldos actuales en variables	`saldoInicialEmisor` y `saldoInicialReceptor` guardados en `store`.	2,000.00 / 2,000.00
4. Si el usuario ya existe, el escenario debe caerse con error	El registro exige `status 201`: si el teléfono ya existiera, la API responde 409 y el escenario falla. Además se comprueba de forma explícita registrando de nuevo al Emisor: `match [400, 409] contains responseStatus` y `code == 'AUTH_006'`.	Duplicado → 409 AUTH_006 "User already exists"

POST http://134.209.211.10:4001/auth/v1/register → 201 · 277 ms

REQUEST


```
{
  "phone": "959477034",
  "fullName": "Emisor QA Karate",
  "pin": "123456",
  "email": "emisor.959477034@qa.test"
}
```


RESPONSE


```
{
  "message": "Usuario creado exitosamente",
  "user": {
    "id": "u_1793234259588",
    "phone": "959477034",
    "name": "Emisor QA Karate",
    "email": "emisor.959477034@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```


POST http://134.209.211.10:4001/auth/v1/register → 201 · 230 ms

REQUEST


```
{
  "phone": "959491530",
  "fullName": "Receptor QA Karate",
  "pin": "123456",
  "email": "receptor.959491530@qa.test"
}
```


RESPONSE


```
{
  "message": "Usuario creado exitosamente",
  "user": {
    "id": "u_1793234259834",
    "phone": "959491530",
    "name": "Receptor QA Karate",
    "email": "receptor.959491530@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```


POST http://134.209.211.10:4001/auth/v1/register → 409 · 213 ms

REQUEST


```
{
  "phone": "959477034",
  "fullName": "Emisor Duplicado",
  "pin": "123456"
}
```


RESPONSE


```
{
  "code": "AUTH_006",
  "message": "User already exists"
}
```

POST http://134.209.211.10:4004/admin/v1/login → 200 · 214 ms

REQUEST

```
{
  "username": "admin",
  "password": "********"
}
```

RESPONSE

```
{
  "token": "eyJhbGciOiJI…",
  "type": "Bearer",
  "expiresIn": 14400,
  "message": "Usa este token en el Header 'Authorization'"
}
```

POST http://134.209.211.10:4004/admin/v1/balance → 200 · 252 ms

REQUEST

```
{
  "userId": "u_1793234259588",
  "amount": 2000,
  "type": "CREDIT",
  "description": "Saldo inicial QA"
}
```

RESPONSE

```
{
  "success": true,
  "newBalance": 2000
}
```

POST http://134.209.211.10:4004/admin/v1/balance → 200 · 1123 ms

REQUEST

```
{
  "userId": "u_1793234259834",
  "amount": 2000,
  "type": "CREDIT",
  "description": "Saldo inicial QA"
}
```

RESPONSE

```
{
  "success": true,
  "newBalance": 2000
}
```

POST http://134.209.211.10:4001/auth/v1/login → 200 · 225 ms

REQUEST


```
{
  "phone": "959477034",
  "pin": "123456"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "expiresIn": 3600,
  "user": {
    "id": "u_1793234259588",
    "phone": "959477034",
    "name": "Emisor QA Karate",
    "email": "emisor.959477034@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```


GET http://134.209.211.10:4002/wallet/v1/balance → 200 · 216 ms

RESPONSE


```
{
  "balance": 2000,
  "currency": "USD"
}
```


POST http://134.209.211.10:4001/auth/v1/login → 200 · 226 ms

REQUEST


```
{
  "phone": "959491530",
  "pin": "123456"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "expiresIn": 3600,
  "user": {
    "id": "u_1793234259834",
    "phone": "959491530",
    "name": "Receptor QA Karate",
    "email": "receptor.959491530@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```


GET http://134.209.211.10:4002/wallet/v1/balance → 200 · 212 ms

RESPONSE


```
{
  "balance": 2000,
  "currency": "USD"
}
```

Escenario 2: Transferencia del Usuario Emisor al Receptor PASS

Paso del enunciado	Implementación (Karate)	Evidencia de esta ejecución
1. Login del Emisor y guardar el token	`POST /auth/v1/login`; `def tokenEmisor = response.token`.	200, token JWT obtenido
2. Transferencia con nota "Pago de Servicios" por 450.00	`POST /transaction/v1/transfer` con `recipientId` del Receptor, `amount: montoTransferencia` y `description: notaTransferencia`.	200 · 450.00 · "Pago de Servicios"
3. Validar la transacción	Esquema con `#notnull` y tabla de aserciones: `status == 'COMPLETED'`, mensaje, `transactionId` (`#regex tx_\d+`), `referenceCode` (`#regex REF-\d+`).	status COMPLETED
4. Guardar el código de la transacción	`store.put('codigoTransaccion', referenceCode)` y `store.put('transactionId', ...)`.	REF-449329 / tx_1793234263209
5. Consultar el saldo del Emisor	`GET /wallet/v1/balance` con `Authorization: Bearer tokenEmisor`.	200
6. Validar la disminución contra el saldo inicial	`saldoEsperado = saldoInicialEmisor - montoTransferencia`; `match response.balance == saldoEsperado`.	2,000.00 − 450.00 = 1,550.00 ✓

POST http://134.209.211.10:4001/auth/v1/login → 200 · 229 ms

REQUEST


```
{
  "phone": "959477034",
  "pin": "123456"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "expiresIn": 3600,
  "user": {
    "id": "u_1793234259588",
    "phone": "959477034",
    "name": "Emisor QA Karate",
    "email": "emisor.959477034@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```

POST http://134.209.211.10:4003/transaction/v1/transfer → 200 · 322 ms

REQUEST


```
{
  "recipientId": "u_1793234259834",
  "amount": 450,
  "description": "Pago de Servicios",
  "categoryId": "cat_tx"
}
```


RESPONSE


```
{
  "transactionId": "tx_1793234263209",
  "referenceCode": "REF-449329",
  "status": "COMPLETED",
  "message": "Transferencia realizada con éxito"
}
```


GET http://134.209.211.10:4002/wallet/v1/balance → 200 · 218 ms

RESPONSE


```
{
  "balance": 1550,
  "currency": "USD"
}
```

Escenario 3: Validación de Transferencia en Receptor PASS

Paso del enunciado	Implementación (Karate)	Evidencia de esta ejecución
1. Login del Receptor y guardar el token	`POST /auth/v1/login`; `def tokenReceptor = response.token`.	200, token JWT obtenido
2. Consultar el saldo del Receptor	`GET /wallet/v1/balance` con el token del Receptor.	200
3. Validar el incremento contra el saldo inicial	`saldoEsperado = saldoInicialReceptor + montoTransferencia`; `match response.balance == saldoEsperado`.	2,000.00 + 450.00 = 2,450.00 ✓
4. Consultar las transacciones del Receptor	`GET /wallet/v1/transactions`; `match each response contains {...}`.	200, lista de movimientos
5. Validar el número de transacción contra el del Escenario 2	`match response[*].referenceCode contains codigoTransaccion`; el movimiento encontrado es `INCOME` de 450.00 con la contraparte y la nota correctas. (El `transactionId` no se persiste en el historial: ver hallazgo H-01).	REF-449329 encontrado ✓

POST http://134.209.211.10:4001/auth/v1/login → 200 · 230 ms

REQUEST


```
{
  "phone": "959491530",
  "pin": "123456"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "expiresIn": 3600,
  "user": {
    "id": "u_1793234259834",
    "phone": "959491530",
    "name": "Receptor QA Karate",
    "email": "receptor.959491530@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```


GET http://134.209.211.10:4002/wallet/v1/balance → 200 · 216 ms

RESPONSE


```
{
  "balance": 2450,
  "currency": "USD"
}
```

GET http://134.209.211.10:4002/wallet/v1/transactions → 200 · 219 ms

RESPONSE


```
[
  {
    "id": "tx-1793234263180-31",
    "referenceCode": "REF-449329",
    "type": "INCOME",
    "amount": 450,
    "date": "2026-10-29T00:37:43.180Z",
    "counterparty": "Dinero Recibido",
    "counterpartyPhone": "959477034",
    "description": "De: 959477034. Pago de Servicios",
    "categoryId": "cat_5",
    "categoryName": "Sueldo"
  },
  {
    "id": "tx-1793234260802-403",
    "referenceCode": "ADM-1793234260798",
    "type": "INCOME",
    "amount": 2000,
    "date": "2026-10-29T00:37:40.803Z",
    "counterparty": "Dinero Recibido",
    "counterpartyPhone": "ADMIN",
    "description": "ADMIN ADJUSTMENT: Saldo inicial QA",
    "categoryId": "cat_tx",
    "categoryName": "Transferencia"
  }
]
```

Feature 3: Flujo Administrador PASS

Paso del enunciado	Implementación (Karate)	Evidencia de esta ejecución
Login con rol Administrador	`POST /admin/v1/login` con `x-channel: ADMIN`; se valida token Bearer y `expiresIn`; con clave inválida → 401 AUTH_003.	200 · 401 con credenciales inválidas
1. Validar la transacción de la operación	`GET /admin/v1/transactions`: la referencia aparece con exactamente 2 asientos — débito `EXPENSE` del Emisor y crédito `INCOME` del Receptor — por 450.00 con las contrapartes cruzadas (tabla de aserciones).	REF-449329: 2 asientos ✓
Validación complementaria de saldos	`GET /admin/v1/users`: Emisor y Receptor `ACTIVE` con los saldos finales.	1,550.00 / 2,450.00 ✓
Validación complementaria de seguridad	`Scenario Outline`: auditoría sin token → 401 ADM_001; con token de usuario → 403 ADM_002.	401 / 403 ✓

ESCENARIO: LOGIN DE ADMINISTRADOR CON CREDENCIALES VALIDAS E INVALIDAS · **PASS**

POST http://134.209.211.10:4004/admin/v1/login → 200 · 216 ms

REQUEST

```
{
  "username": "admin",
  "password": "********"
}
```

RESPONSE

```
{
  "token": "eyJhbGciOiJI…",
  "type": "Bearer",
  "expiresIn": 14400,
  "message": "Usa este token en el Header 'Authorization'"
}
```

POST http://134.209.211.10:4004/admin/v1/login → 401 · 216 ms

REQUEST

```
{
  "username": "admin",
  "password": "********"
}
```

RESPONSE

```
{
  "code": "AUTH_003",
  "message": "Invalid admin credentials"
}
```

ESCENARIO: EL ADMINISTRADOR VALIDA LA TRANSACCION EN LA AUDITORIA GLOBAL · **PASS**

POST http://134.209.211.10:4004/admin/v1/login → 200 · 216 ms

REQUEST


```
{
  "username": "admin",
  "password": "********"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "type": "Bearer",
  "expiresIn": 14400,
  "message": "Usa este token en el Header 'Authorization'"
}
```


GET http://134.209.211.10:4004/admin/v1/transactions → 200 · 1137 ms

RESPONSE


```
[
  {
    "id": "tx-1790119463453-881",
    "referenceCode": "ADM-1790119463445",
    "type": "INCOME",
    "amount": 1000,
    "date": "2026-09-22T23:24:23.453Z",
    "counterparty": "Dinero Recibido",
    "counterpartyPhone": "ADMIN",
    "description": "ADMIN ADJUSTMENT: Bono",
    "categoryId": "cat_tx",
    "categoryName": "Transferencia",
    "userId": "u1"
  },
  {
    "id": "dep-1785210786064",
    "referenceCode": "DEP-1785210786064",
    "type": "INCOME",
    "amount": 500,
    "date": "2026-07-28T03:53:06.064Z",
    "counterparty": "Recarga Bancaria",
    "counterpartyPhone": "BANCO",
    "description": "Carga de saldo manual",
    "categoryId": "cat_inc_1",
    "categoryName": "Sueldo",
    "userId": "u1"
  }
]
… (3410 elementos en total)
```


ESCENARIO: EL ADMINISTRADOR CONFIRMA LOS SALDOS FINALES DE EMISOR Y RECEPTOR · **PASS**

POST http://134.209.211.10:4004/admin/v1/login → 200 · 214 ms

REQUEST


```
{
  "username": "admin",
  "password": "********"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "type": "Bearer",
  "expiresIn": 14400,
  "message": "Usa este token en el Header 'Authorization'"
}
```


GET http://134.209.211.10:4004/admin/v1/users → 200 · 7336 ms

RESPONSE


```
[
  {
    "id": "u1",
    "phone": "999888777",
    "name": "Deyvi Emisor",
    "email": "derojas@test.com",
    "pin": "123456",
    "status": "DELETED",
    "contacts": [
      "u2",
      "u_1785206408042"
    ],
    "balance": 1.0000104105245205e+25
  },
  {
    "id": "u2",
    "phone": "666555444",
    "name": "Beto Builder",
    "email": "beto@wallet.com",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": [],
    "balance": 3407
  }
]
… (3648 elementos en total)
```


ESCENARIO: ACCESO A LA AUDITORIA SIN PERMISOS DE ADMINISTRADOR - SIN TOKEN · **PASS**

POST http://134.209.211.10:4001/auth/v1/login → 200 · 214 ms

REQUEST


```
{
  "phone": "959477034",
  "pin": "123456"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "expiresIn": 3600,
  "user": {
    "id": "u_1793234259588",
    "phone": "959477034",
    "name": "Emisor QA Karate",
    "email": "emisor.959477034@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```


GET http://134.209.211.10:4004/admin/v1/transactions → 401 · 210 ms

RESPONSE


```
{
  "code": "ADM_001",
  "message": "Admin token missing"
}
```


ESCENARIO: ACCESO A LA AUDITORIA SIN PERMISOS DE ADMINISTRADOR - TOKEN DE USUARIO SIN ROL · **PASS**

POST http://134.209.211.10:4001/auth/v1/login → 200 · 216 ms

REQUEST


```
{
  "phone": "959477034",
  "pin": "123456"
}
```


RESPONSE


```
{
  "token": "eyJhbGciOiJI…",
  "expiresIn": 3600,
  "user": {
    "id": "u_1793234259588",
    "phone": "959477034",
    "name": "Emisor QA Karate",
    "email": "emisor.959477034@qa.test",
    "pin": "123456",
    "status": "ACTIVE",
    "contacts": []
  }
}
```

GET http://134.209.211.10:4004/admin/v1/transactions → 403 · 211 ms

RESPONSE


```
{
  "code": "ADM_002",
  "message": "Unauthorized admin access"
}
```

Reporte de Karate – regression.feature

Karate Labs

3

0

qa

Scenarios

2026-10-28 07:37:44 p. m.

[1:26] Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado

[2:102] Escenario 2 - Transferencia del Emisor al Receptor por 450.00

[3:155] Escenario 3 - Validacion del saldo y del historial en el Receptor

Summary | Tags | Feature: ***features/regression.feature*** | **Regresion Billetera Digital - Registro, transferencia y validacion en receptor** *Flujo E2E sobre los microservicios Auth (4001), Wallet (4002), Transaction (4003) y Admin (4004). Los datos se comparten entre escenarios mediante el almacen global "store" definido en karate-config.js, por lo que el orden importa y la ejecucion es secuencial (parallel(1)).*

regression parallel=false escenario1

Scenario: [1:26] **Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado** — ms: 3740

>>	Background:	
	# ---------- Datos de prueba ----------	
28	* def emisor = { phone: '#(generarCelular())', fullName: 'Emisor QA Karate', pin: '#(pinDefault)' }	17
29	* set emisor.email = 'emisor.' + emisor.phone + '@qa.test'	3
30	* def receptor = { phone: '#(generarCelular())', fullName: 'Receptor QA Karate', pin: '#(pinDefault)' }	2
31	* set receptor.email = 'receptor.' + receptor.phone + '@qa.test'	1
32	* match emisor.phone == '#regex 9\\d{8}'	10
33	* match receptor.phone == '#regex 9\\d{8}'	1
34	* assert emisor.phone != receptor.phone	3
	# ---------- Registro Emisor ----------	
37	Given url authUrl	0
38	And path 'register'	0
39	And request emisor	0
40	When method post	634
41	Then status 201	0
42	And match response.message == 'Usuario creado exitosamente'	1
43	And match response.user == esquemaUsuario	1
44	And match response.user contains { phone: '#(emisor.phone)', name: '#(emisor.fullName)', email: '#(emisor.email)' }	2
45	* set emisor.id = response.user.id	1
	# ---------- Registro Receptor ----------	
48	Given url authUrl	0
49	And path 'register'	0
50	And request receptor	0
51	When method post	233
52	Then status 201	0
53	And match response.user == esquemaUsuario	1
54	And match response.user contains { phone: '#(receptor.phone)', name: '#(receptor.fullName)', email: '#(receptor.email)' }	2
55	* set receptor.id = response.user.id	1
	# ---------- Validacion negativa: usuario duplicado ----------	
58	Given url authUrl	0
59	And path 'register'	0
60	And request { phone: '#(emisor.phone)', fullName: 'Emisor Duplicado', pin: '#(pinDefault)' }	0
61	When method post	214
62	Then match [400, 409] contains responseStatus	4
63	And match response == { code: 'AUTH_006', message: '#string' }	0
	# ---------- Asignacion de saldo (Admin Service, llamada en bucle) ----------	
66	* def admin = call read('classpath:features/common/admin-login.feature')	230
>>	features.common.admin-login	218
67	* table abonos	5

userId	amount	adminToken
emisor.id	saldoAsignado	admin.token
receptor.id	saldoAsignado	admin.token

71	* call read('classpath:features/common/asignar-saldo.feature') abonos	1398
>>	[0]features.common.asignar-saldo	258
>>	[1]features.common.asignar-saldo	1127
	# ---------- Verificacion del saldo con el token de cada usuario ----------	
74	* def sesionEmisor = call read('classpath:features/common/login.feature') { phone: '#(emisor.phone)', pin: '#(emisor.pin)' }	239
>>	features.common.login	232
75	Given url walletUrl	0

Reporte de Karate – admin.feature

Karate Labs

5

0

qa

Scenarios

2026-10-28 07:37:56 p. m.

[1:14] Login de administrador con credenciales validas e invalidas

[2:31] El administrador valida la transaccion en la auditoria global

[3:66] El administrador confirma los saldos finales de Emisor y Receptor

[4.1:105] Acceso a la auditoria sin permisos de administrador - sin token

[4.2:106] Acceso a la auditoria sin permisos de administrador - token de usuario sin rol

Summary | Tags | Feature: ***features/admin.feature*** | **Feature 3 - Flujo Administrador: auditoria de la transaccion** *Usa los datos que regression.feature dejo en el almacen global (usuarios, saldos y codigo de transaccion) para validar la operacion desde el backoffice.*

admin parallel=false admin-login — ms: 443

Scenario: [1:14] **Login de administrador con credenciales validas e invalidas**

>>	Background:	
15	Given url adminUrl	0
16	And path 'login'	0
17	And request adminCredentials	0
18	When method post	218
19	Then status 200	0
20	And match response == { token: '#notnull', type: 'Bearer', expiresIn: '#number', message: '##string' }	0
21	And match response.expiresIn == 14400	1
23	Given url adminUrl	0
24	And path 'login'	0
25	And request { username: '#(adminCredentials.username)', password: 'clave-incorrecta' }	0
26	When method post	218
27	Then status 401	0
28	And match response == { code: 'AUTH_003', message: '#string' }	0

admin parallel=false admin-auditoria — ms: 1515

Scenario: [2:31] **El administrador valida la transaccion en la auditoria global**

>>	Background:	
32	* def admin = call read('classpath:features/common/admin-login.feature')	221
>>	features.common.admin-login	217
34	Given url adminUrl	0
35	And path 'transactions'	0
36	And header Authorization = 'Bearer ' + admin.token	0
37	When method get	1228
38	Then status 200	0
39	And match response == '#[_ > 0]'	2
40	And match response[*].referenceCode contains codigoTransaccion	16
	# La operacion genera dos asientos con la misma referencia: debito y credito	
43	* def asientos = karate.filter(response, function(t){ return t.referenceCode == codigoTransaccion })	37
44	* match asientos == '#[2]'	2
45	* def debito = karate.filter(asientos, function(t){ return t.type == 'EXPENSE' })[0]	1
46	* def credito = karate.filter(asientos, function(t){ return t.type == 'INCOME' })[0]	1
48	* table asercionesAsiento	1

asiento	userId	type	contraparte
debito	emisor.id	'EXPENSE'	receptor.phone
credito	receptor.id	'INCOME'	emisor.phone

52	* def validar =	1
60	* def resultados = karate.map(asercionesAsiento, validar)	5
61	* match resultados == [true, true]	0
62	* match debito.description contains notaTransferencia	0
63	* match credito.description contains notaTransferencia	0

admin parallel=false admin-usuarios — ms: 7684

Scenario: [3:66] **El administrador confirma los saldos finales de Emisor y Receptor**

>>	Background:	
67	* def admin = call read('classpath:features/common/admin-login.feature')	224
>>	features.common.admin-login	220
68	* def saldoFinalEmisor = store.get('saldoFinalEmisor')	1
69	* def saldoFinalReceptor = store.get('saldoFinalReceptor')	0
71	Given url adminUrl	0

4. Hallazgos

ID	Severidad	Endpoint	Descripción	Recomendación
H-01	**Media**	`POST /transaction/v1/transfer` · `GET /wallet/v1/transactions`	El **transactionId** devuelto por la transferencia no aparece en el historial; los movimientos tienen otros IDs (`tx-<timestamp>-<n>`). El único dato común es **referenceCode**.	Persistir el transactionId en ambos asientos o documentar referenceCode como identificador oficial. La suite valida la trazabilidad por referenceCode.
H-02	**Alta**	`POST /auth/v1/register` · `POST /auth/v1/login` · `GET /admin/v1/users`	Las respuestas exponen el **PIN en texto plano** (`"pin":"123456"`).	No devolver el PIN en ninguna respuesta y almacenarlo con hash.
H-03	**Alta**	`POST /admin/v1/balance`	El ajuste de saldo no tiene límites: en el entorno existen usuarios con saldo `1.0e+72` y `-1.0e+72` (montos desmesurados y saldos negativos).	Validar el rango del monto y no permitir saldo final negativo.
H-04	**Media**	`POST /auth/v1/register`	No se valida el formato del teléfono ni del email: hay usuarios con teléfono `ASDASDASD`, `123` o de 16 dígitos, y emails como `9191com`.	Validar teléfono (9 dígitos numéricos) y email en el backend y responder 400.
H-05	**Baja**	`PUT /admin/v1/users/:id`	Se aceptan estados fuera del catálogo (`ACTIVEeeee`).	Restringir status a ACTIVE / DELETED.
H-06	**Baja**	`POST /admin/v1/login`	La documentación indica que el Admin Service solo acepta `x-channel: ADMIN`, pero el login responde 200 también con `x-channel: WEB`.	Rechazar con 400 GEN_001 los canales distintos de ADMIN.

H-03, H-04 y H-05 se observaron en datos del entorno compartido creados por otras ejecuciones; demuestran que la API aceptó esos valores.

5. Conclusiones

1. El flujo principal de la billetera **cumple** lo esperado: 8 de 8 escenarios aprobados. Los saldos cuadran exactamente (−450.00 en el Emisor, +450.00 en el Receptor) y la operación queda registrada con doble asiento en la auditoría del administrador.
2. Los controles funcionan: registro duplicado → 409 AUTH_006; Admin sin token → 401 ADM_001; Admin con token de usuario → 403 ADM_002; credenciales de admin inválidas → 401 AUTH_003.
3. La trazabilidad de la transferencia depende de **referenceCode**, porque el transactionId devuelto no aparece en el historial (H-01).

4. Existen riesgos de **seguridad** (PIN expuesto, H-02) y de **integridad de datos** (montos y formatos sin validar, H-03 a H-05) que deben corregirse antes de pasar a producción.
5. La suite es repetible: genera usuarios con teléfonos únicos en cada corrida, no depende de datos previos y se ejecuta con `mvn clean test` en unos 19 segundos.

Próximos pasos sugeridos: casos negativos de transferencia (saldo insuficiente WALL_001, límite diario WALL_002, destinatario inexistente TRX_002, autotransferencia TRX_001), integración en CI y ejecución sobre un entorno con datos aislados.

Informe generado el 28/10/2026 19:58 a partir de `target/karate-reports`.